

Excellent! This is one of the most important topics in AI. Once you understand this lesson, you'll understand how **Word2Vec**, **BERT**, **Sentence Transformers**, **OpenAI Embeddings**, and even **ChatGPT** represent words internally.

Today we'll learn **how a word becomes numbers**.

Step 2 – One-Hot Encoding → Embedding Layer → Word Vector

Goal

We want the computer to understand this word:

cat



A computer **cannot understand text directly**.

It only understands numbers.

So we need to convert:

cat



into numbers.

Our Small Vocabulary

Suppose we have only five words.

Index	Word
0	cat
1	dog
2	milk
3	water
4	car

The computer stores them like this.

0 → cat



1 → dog

2 → milk

3 → water

4 → car

Step 1 – One-Hot Encoding

One-Hot Encoding creates a vector where:

- The current word = **1**
- All other words = **0**

Example:

cat

[1,0,0,0,0]



dog

[0,1,0,0,0]



milk

[0,0,1,0,0]



water

[0,0,0,1,0]



car

```
[0,0,0,0,1]
```



Why is it called "One-Hot"?

Because only **one position** is hot (1).

Everything else is zero.

Problem with One-Hot Encoding

Suppose we compare:

cat



↓

```
[1,0,0,0,0]
```

and

dog



↓

```
[0,1,0,0,0]
```

The computer thinks they are completely different.

Likewise,

car



↓

```
[0,0,0,0,1]
```

is also completely different.

But humans know:

cat



dog

are much more similar than:

cat



car

One-Hot Encoding cannot capture meaning.

Large Vocabulary Problem

Suppose your vocabulary contains:

100,000 words



Then:

cat



becomes

[1,0,0,0,0,0,0,0,0,0,...100000 values]



Most of the values are **0**.

This is called a **sparse vector**.

Sparse vectors:

- Waste memory
 - Waste CPU time
 - Don't capture meaning
-

The Solution – Embedding Layer

Instead of storing this:

cat



↓

[1,0,0,0,0]

we create a small table of learned vectors.

For example (using only 3 dimensions for illustration):

Word	Embedding
cat	[0.81,0.22,0.45]
dog	[0.79,0.24,0.43]
milk	[-0.12,0.91,0.31]
water	[-0.10,0.94,0.28]
car	[0.50,-0.71,0.84]

These vectors are **learned during training**, not written by a programmer.

Notice Something Amazing

Look at:

cat



↓

[0.81,0.22,0.45]

and

dog



↓

[0.79,0.24,0.43]

They are very close.

Now compare:

cat



↓

[0.81, 0.22, 0.45]

and

car



↓

[0.50, -0.71, 0.84]

Very different.

The model has learned semantic similarity.

Where Do These Numbers Come From?

Initially:

cat



↓

[0.13, -0.42, 0.88]

Random.

Dog:

[-0.77, 0.29, -0.12]



Also random.

After reading millions of sentences:

The cat drinks milk.



The dog drinks milk.

The cat eats fish.

The dog eats meat.

The neural network updates the vectors over and over until similar words end up close together.

What Does the Embedding Layer Look Like?

Imagine it as a table:

Embedding Layer

Word	X1	X2	X3
cat	0.81	0.22	0.45
dog	0.79	0.24	0.43
milk	-0.12	0.91	0.31
water	-0.10	0.94	0.28
car	0.50	-0.71	0.84

When the input is:

dog

the embedding layer simply returns:

[0.79, 0.24, 0.43]

How Does the Neural Network Learn These Values?

Suppose the sentence is:

The cat drinks milk.

Using Skip-Gram:

Input:

cat

Expected output:

The

drinks

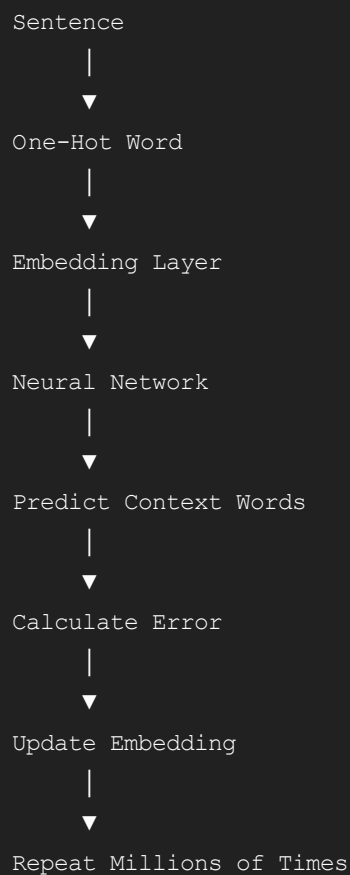
milk

Initially, the prediction is poor.

The network computes the error and adjusts the embedding for **cat** slightly.

This happens millions of times for all words.

The learning loop is:



Over time, the embeddings become meaningful.

Real Embedding Sizes

The examples above used only **3 numbers** to make them easy to understand.

Real models use much larger vectors:

Model	Dimensions
Word2Vec	100–300
GloVe	50–300

Model	Dimensions
FastText	100–300
BERT Base	768
BERT Large	1024
OpenAI text-embedding-3-small	1536

More dimensions allow the model to represent more nuanced meaning.

Can You Build This in C#?

Yes, but **not from scratch** unless your goal is to study machine learning algorithms.

For practical applications in C#:

- You typically **use a pre-trained embedding model**.
- You generate embeddings for your documents and queries.
- You compare them with **cosine similarity**.
- That forms the retrieval stage of a modern **RAG chatbot**.

Your Learning Progress

You have now completed the theory behind:

Traditional NLP

- ✓ Tokenization
- ✓ Stop Words
- ✓ Stemming
- ✓ Lemmatization
- ✓ Bag of Words
- ✓ TF
- ✓ IDF
- ✓ TF-IDF
- ✓ Cosine Similarity

Modern NLP

- ✓ Word2Vec

Next Lesson

The next topic is **Sentence Embeddings**, where you'll learn why modern systems don't embed words individually anymore. Instead, they create **one vector for an entire sentence or paragraph**, which is the key technology behind semantic search, vector databases, and RAG systems. That is the natural next step before building a real AI-powered chatbot in C#.